

GIT

Best practices for collaborative development

by Francesco Agnoletto

WHAT IS VERSION CONTROL?

Version control is the software engineering practice of **controlling**, **organizing**, and **tracking** different versions in history of computer files.

Version control systems offer many benefits to any project:

- Record the history of a project.
- Collaborate asynchronously.
- Track changes.
- Time travel.

Git is an **open source** and **distributed** version control system created by Linus Torvalds in 2005.

Git's core design principles:

- Decentralized.
- Data integrity.
- Offline first.
- Fast.

REMOTE

The remote is the version of the repository **hosted on a server** (usually GitHub or GitLab)

origin is usually the default remote where fetch and push operations direct.

```
$ git remote -v  
  
> origin  git@github.com:Kornil/my-repo.git (fetch)  
> origin  git@github.com:Kornil/my-repo.git (push)
```


Git stores a **local copy** of remote branches.

Updating them does not impact your working branches.

```
$ git fetch -p  
  
# origin/master is updated  
# master is untouched
```

The extra **-p** will also prune stale remote-tracking branches.

We can also update our local references with `git pull`.

This will also update our local branch with the latest commits from the remote.

```
$ git pull -p  
  
# origin/master is updated  
# master is also updated
```

`git pull` can also be pointed at a different branch.

`git pull origin master` is a useful way to update a `feature branch` with the latest code from `master`.

COMMITTS

The commit is Git's fundamental unit, it is **unique** and **immutable**.

Commits record 5 essential details:

- Commit SHA-1 hash.
- Parent commit hash.
- Message title and body.
- Author name and email.
- Date.

INTERLUDE #1: THE 3 STATES

WORKING DIR

Actual files on your machine.

unversioned changes

STAGING AREA

Temporary area to build
snapshots for commits.

changes ready to commit

git add →

GIT REPOSITORY

Snapshots of files held in git as
commits.

committed snapshots

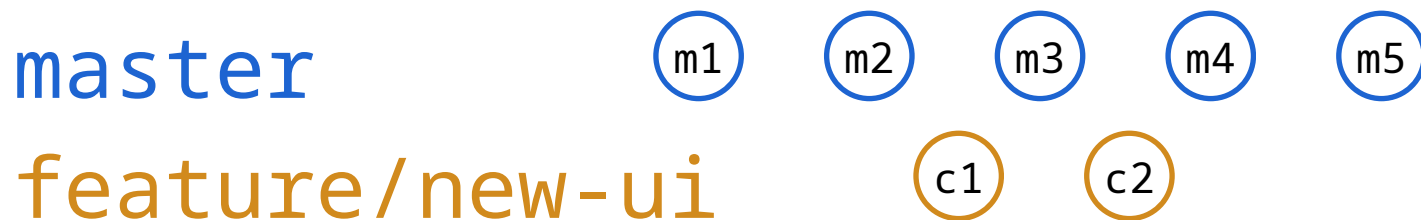
git commit →

BRANCHES

A branch is a movable **pointer** to a specific commit.

It creates an **independent line of development**.

By nature, branches are very cheap to create



Establishing a **consistent naming convention** for branches is a good practice for collaborative development. It directly improves:

- Clarity of intent.
- Traceability.
- Organization.
- Scalability.
- Efficiency.

HEAD

HEAD is a special pointer indicating the **current commit** where you are.

HEAD is a special pointer indicating the **current commit** where you are.

By default, HEAD always points at the **latest commit** of the current branch.

HEAD is a special pointer indicating the **current commit** where you are.

By default, HEAD always points at the **latest commit** of the current branch.

It can be manually **detached** to point at any commit.

HEAD is a special pointer indicating the **current commit** where you are.

By default, HEAD always points at the **latest commit** of the current branch.

It can be manually **detached** to point at any commit.

Never commit in a detached head.

You will lose your work.

INTERLUDE #2: STASH

The stash can be used to **store uncommitted changes** and clean up the working directory.

This allows to quickly **switch between different branches** without having to commit WIP code.

```
# Store uncommitted changes
$ git stash push -m "new route wip"

$ git stash list
> stash@{0}: On master: new route wip

# Retrieve previously stored changes and delete the stash
$ git stash pop stash@{0}

# Retrieve previously stored changes and keep the stash
$ git stash apply stash@{0}
```

BRANCHING STRATEGIES

There are **a few strategies** when working with git, each with its own advantages and disadvantages.

TRUNK-BASED DEVELOPMENT

- Single branch.
- Small, frequent commits.
- Feature flags.

trunk



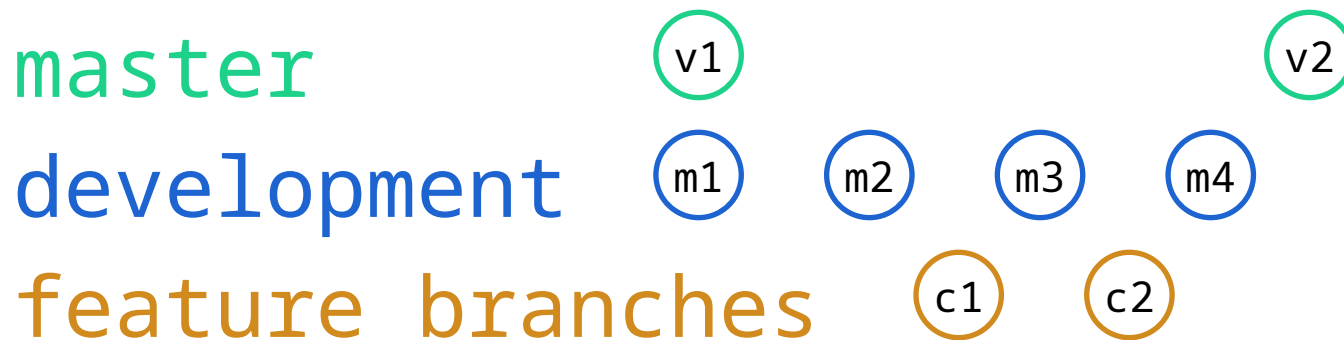
GITHUB FLOW

- Single main branch.
- Short-lived branches.
- CI/CD.



GITFLOW

- Multiple primary branches.
- Strict rules.
- Structured releases.



GITHUB FLOW

Pros:

- Fast development.
 - Less complexity.
 - More transparent.
-

Cons:

- Reliance on tests.
- Minimal release versioning.
- Less enforced discipline.

GITFLOW

Pros:

- Clear versions.
 - Isolated development.
 - Structured.
-

Cons:

- Slow.
- Complex CI/CD.
- Complex in general.

THE SINGLE PURPOSE PRINCIPLE

Each commit should have a **single scope**.

Each commit should have a **single scope**.

The commit title should be **clear** and **concise**, extra information can be displayed in the body.

Each commit should have a **single scope**.

The commit title should be **clear** and **concise**, extra information can be displayed in the body.

```
$ git commit -m "Add createDashboard button"
```

```
$ git commit -m "Fix #123" -m "Additional details here"
```

Each commit should have a **single scope**.

The commit title should be **clear** and **concise**, extra information can be displayed in the body.

```
$ git commit -m "Add createDashboard button"
```

```
$ git commit -m "Fix #123" -m "Additional details here"
```

Single lines of code can be staged by using `git add -p` or any modern code editor.

Each branch should contain a **single feature** or **bugfix**.

Each branch should contain a **single feature** or **bugfix**.

```
$ git checkout -b "feature/grid-drag-drop"
```

```
$ git checkout -b "refactor/grid-module-typescript"
```

```
$ git checkout -b "fix/issue-33"
```

Atomic commits and branches have many benefits:

Atomic commits and branches have many benefits:

- Easier to review.

Atomic commits and branches have many benefits:

- Easier to review.
- Cleaner history.

Atomic commits and branches have many benefits:

- Easier to review.
- Cleaner history.
- Easy to debug.

Atomic commits and branches have many benefits:

- Easier to review.
- Cleaner history.
- Easy to debug.
- Easy to revert.

SHARED VS. LOCAL HISTORY

*“ You must never EVER destroy other peoples history.
Once you've published your history in some public site,
other people may be using it, and so now it's clearly not
your private history any more. ”*

— Linus Torvalds

The key distinctions between **Local** vs. **Shared** history:

The key distinctions between **Local** vs. **Shared** history:

- You **can** modify history that is only on your machine in a private branch.

The key distinctions between **Local** vs. **Shared** history:

- You **can** modify history that is only on your machine in a private branch.
- You **should not** modify history that has already been published, even if it's your own branch.

Git history, especially once it's shared, should be treated as a **public record**.

The consequences of not preserving history:

- Breaks team collaboration.
- Loss of auditability.
- Breaks trust and wastes time.

HOW TO SAFELY ALTER HISTORY

While we don't want to alter history, there are **safe ways to delete commits** before or after we publish them.

GIT REVERT

The **easiest** and **safest** way to delete a commit.

It will generate a new commit opposite of the target commit, removing all its changes and recording it in git's history.

```
$ git revert commit_hash_to_delete
```

```
$ git log -2
```

```
commit new_commit_hash  
  author: me  
  date: now  
  revert "bad commit"
```

```
commit commit_hash_to_delete  
  author: me  
  date: now  
  bad commit
```

GIT RESET

This should be **only used on private branches**, it will modify history.

In this example only the last commit is modified but HEAD can be moved to affect more commits.

```
# last commit is gone, its content is now in the staging area  
$ git reset --soft HEAD~
```

```
# last commit is gone, its content is now in the working area  
$ git reset HEAD~
```

```
# last commit is gone, its content is gone  
$ git reset --hard HEAD~
```

GIT REBASE

This is a more "complete" command compared to reset, should be **only used on private branches**, it will modify history.

```
# will target the last 4 commits
$ git rebase -i HEAD~4

# will open a rebase editor
pick 1234 Add readme.md
pick 2345 Fix typo
pick 3456 Fix another typo
pick 4567 work in progress
pick 5678 Add deploy script
pick 6789 Add second deploy script
```


Most common commands:

- **pick** will keep a commit as is.
- **fixup** will merge a commit to the previous one, discarding its commit message.
- **drop** will delete a commit.
- **reword** will rename a commit.
- **squash** will merge a commit to the previous one and combine the messages.

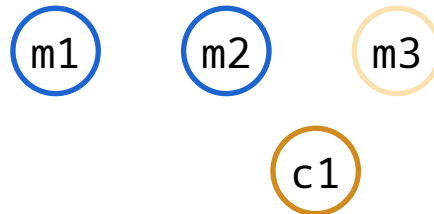
```
# rebase editor
pick 1234 Add readme.md
fixup 2345 Fix typo
drop 3456 Fix another typo
reword 4567 work in progress
pick 5678 Add deploy script
squash 6789 Add second deploy script
```

MERGE VS REBASE

The main branch moves on while you work on a feature. Integrating these changes is vital to keep our work updated.

main

feature



Your choice depends on your goal for the project's history.

GIT MERGE

Preserves historical context.

Keeps a record of when a branch was merged.

Best for maintaining an audit trail.

GIT REBASE

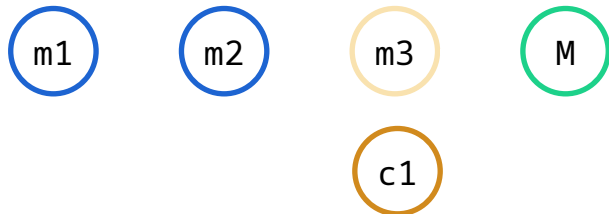
Maintains a linear history.

Creates a cleaner history, easier to read and navigate.

Best for local cleanup before sharing.

The resulting history is fundamentally different.

MERGE HISTORY



REBASE HISTORY



INTERLUDE #3: PULL REQUESTS

A **pull request** or **merge request** is a proposal to merge changes from one branch into another.

It is not a part of git.

The pull request is one of the core features that led the rise of platforms like GitHub, it improves some big pain points of git:

- Code review.
- Discussion.
- Integration.
- Automation.

IDEAL WORKFLOW

There are only a few commands we need to use git effectively.

- git pull
 - `-p` for prune
 - `origin branch-name` for pulling another branch
- git checkout
 - `-b` to create a branch
- git branch
 - `-D branch-name` to delete a branch
- git add/git commit
- git push

WORKING ON A NEW FEATURE FROM SCRATCH

```
$ git checkout master
$ git pull -p

$ git checkout -b feature/download-button

$ git add button.tsx
$ git commit -m "Add button"

$ git add button.test.ts
$ git commit -m "Add button tests" -m "Existence check and cli

$ git push
```

Special mentions

\$ git log -3

\$ git rebase -i HEAD~3

\$ git revert commit-hash

\$ git status

\$ git cherry-pick commit-hash

\$ git fetch -p

We don't use git for ourselves, but for our team. Every operation we do should reflect that, without ego.